Journal of Computational Physics 475 (2023) 111855

Contents lists available at ScienceDirect

Journal of Computational Physics

journal homepage: www.elsevier.com/locate/jcp

# Long-time integration of parametric evolution equations with physics-informed DeepONets

Sifan Wang [a], Paris Perdikaris [b,*]

[a] Graduate Group in Applied Mathematics and Computational Science, University of Pennsylvania, Philadelphia, PA 19104, United States of America

[b] Department of Mechanical Engineering and Applied Mechanics, University of Pennsylvania, Philadelphia, PA 19104, United States of America

ARTICLE INFO

*Article history:*
Received 8 November 2021
Received in revised form 19 October 2022
Accepted 8 December 2022
Available online 19 December 2022

*Keywords:*
Deep learning
Computational science
Differential equations
Dynamical systems

ABSTRACT

Ordinary and partial differential equations (ODEs/PDEs) play a paramount role in analyzing and simulating complex dynamic processes across all corners of science and engineering. In recent years machine learning tools are aspiring to introduce new effective ways of simulating such equations, however existing approaches are not able to reliably return stable and accurate predictions across long temporal horizons. We aim to address this challenge by introducing an effective framework for learning evolution operators that map random initial conditions to associated ODE/PDE solutions within a short time interval. Such operators can be parametrized by deep neural networks that are trained in an entirely self-supervised manner without requiring one to generate any paired input-output observations. Global long-time predictions across a range of initial conditions can be then obtained by iteratively evaluating the trained model using each prediction as the initial condition for the next evaluation step. This introduces a new approach to temporal domain decomposition that is shown to be effective in performing accurate long-time simulations for a wide range of parametric ODE and PDE systems, from wave propagation, to reaction-diffusion dynamics and stiff chemical kinetics, introducing a new way of rapidly emulating non-equilibrium processes in science and engineering.

© 2022 Elsevier Inc. All rights reserved.

## 1. Introduction

Evolution equations, typically expressed as systems of time-dependent ordinary or partial differential equations (ODEs/PDEs), play a prominent role in the analysis, modeling and simulation of complex dynamical systems across diverse scientific domains, from fluid mechanics, to electromagnetics, quantum mechanics, and elasticity [1]. Classical approaches to simulating such equations often assume a discrete, finite-dimensional representation of the unknown solution (typically parametrized by a linear combination of fixed features such as polynomials, trigonometric functions, etc.), the weights of which are inferred by solving large linear or nonlinear systems, depending on the nature of the underlying equation and the scheme used to discretize it [2]. For time-dependent problems these weights vary with time, and can be iteratively updated via appropriate time-integration schemes that discretize the temporal prediction horizon into a small number of steps, the size of which is dictated by the governing time-scales of the problem and the stability properties of the temporal discretization employed [3]. This general workflow has been thoroughly studied over the last several decades, leading to ro-

* Corresponding author.
*E-mail addresses:* sifanw@sas.upenn.edu (S. Wang), pgp@seas.upenn.edu (P. Perdikaris).

https://doi.org/10.1016/j.jcp.2022.111855
0021-9991/© 2022 Elsevier Inc. All rights reserved.

bust and provably accurate techniques such as the finite-element method [4], Runge-Kutta schemes [3], and Krylov subspace methods [5] that serve as the main workhorses of modern computational science and engineering. But as the complexity of the underlying evolution equations increases, so does the cost of simulating them; a cost that quickly becomes unbearable when multiple scenarios need to be queried (e.g., corresponding to different initial and boundary conditions (IBCs), random inputs, forcing terms, etc.), and/or when multi-scale interactions dictate the underlying dynamics.

As the machine learning (ML) revolution is persistently reaching all corners of science, a new wave of techniques are being proposed for accelerating the simulation of ODEs and PDEs [6]. Instead of representing the target solution using a set of fixed features that are determined a-priori, neural networks [7,8] and kernel machines [9–11] offer the possibility of learning effective representations that are adapted to the underlying physical law. Other representative approaches include the deep energy method [12,13], which employs a variational formulation and represents the trial functions by neural networks. Another popular framework is physics-informed neural networks (PINNs) [14] that opts to represent the entire spatio-temporal solution of a PDE system using a single deep neural network that is trained to jointly fit observed data (e.g., IBCs), as well as ensure that the predicted solution satisfies the underlying system of PDEs by minimizing its residual. An attractive property of this approach is that it no longer requires a spatial or temporal discretization of the PDE, nor it requires any external training data (other than knowledge of appropriate IBCs). Moreover, the entire global solution can be rapidly obtained once the network has been trained [14]. However, this remarkable flexibility often comes at the price of reduced accuracy, as well as a multitude of caveats that hinder the training and convergence of such models [15–17]. Nevertheless, PINNs [18], their variants [19–21], and other ML-based approaches [22–24] are currently enjoying increased popularity across diverse applications including fluid mechanics [25,26], heat transfer [27,28], bio-engineering [29,30], materials [31–33], and finance [34,35]. However, a major challenge still remains unsolved and has been largely overlooked in the existing PINNs literature: such self-supervised approaches often fail to accurately simulate evolution equations over a long-time prediction horizon.

In this work we attempt to address this fundamental challenge by leveraging the recently developed framework of physics-informed deep operator networks (DeepONets) [36] to parametrize and learn operators that map random initial conditions to their associated ODE/PDE solutions within a short time interval. The proposed deep learning model can be trained in an entirely self-supervised manner (i.e. without the need for any paired input-output data), only assuming knowledge of the ODE/PDE model form and its associated IBCs. Once the model has been trained on a collection of initial conditions, it can be used to construct the global ODE/PDE solution across a long-time prediction horizon via a simple iterative procedure, in which the model prediction over a short time-step is used as an initial condition for the next evaluation. We demonstrate that this approach can effectively enable rapid long-time emulation of evolution equations subject to a range of initial conditions with good generalization accuracy. Taken together, the computational infrastructure developed in this work can have a broad technical impact in significantly reducing computational costs and accelerating scientific modeling of complex non-linear, non-equilibrium processes across diverse applications.

The remaining of this paper is structured as follows. In section 2, we provide an overview of the PINNs framework put forth by Raissi et al. [14] and demonstrate its fundamental limitations in approximating ODE/PDE solutions over long-time horizons. Section 3 provides a detailed discussion of our main technical contributions, starting with a recap on physics-informed DeepONets [36] in section 3.1, followed by the proposed formulation for tackling long-time integration problems in section 3.2. Further, in section 4 we present a series of comprehensive numerical studies to assess the performance of the proposed long-time integration framework across a range of parametric ODE/PDE systems involving wave propagation, reaction-diffusion dynamics, and stiff chemical kinetics. Finally, section 5 concludes with a discussion of our main findings, potential pitfalls, and shortcomings, as well as future research directions emanating from this study. All code and data accompanying this manuscript will be made available at https://github.com/PredictiveIntelligenceLab/Long-time-Integration-PI-DeepONets.

## 2. Physics-informed neural networks

In this section, we give a brief review of physics-informed neural networks (PINNs) [18] for solving time-dependent ODEs and PDEs. Generally, we consider initial–boundary value problems taking the form

$$\boldsymbol{s}_t + \mathcal{N}_{\boldsymbol{x}}[\boldsymbol{s}] = \boldsymbol{0}, \quad \boldsymbol{x} \in \Omega, t \in [0, T] \tag{2.1}$$

$$\boldsymbol{s}(\boldsymbol{x}, t) = \boldsymbol{g}(\boldsymbol{x}, t), \quad \boldsymbol{x} \in \partial\Omega, t \in [0, T] \tag{2.2}$$

$$\boldsymbol{s}(\boldsymbol{x}, 0) = \boldsymbol{u}(\boldsymbol{x}), \quad \boldsymbol{x} \in \Omega, \tag{2.3}$$

where $\boldsymbol{x}$ and $t$ represent spatial and temporal coordinates, respectively, $\mathcal{N}_{\boldsymbol{x}}$ denotes a differential operator with respect to $\boldsymbol{x}$, and $\Omega \subset \mathbb{R}^n$ is an open, bounded domain with a well-behaved boundary $\partial\Omega$. In addition, $\boldsymbol{s} : \overline{\Omega} \to \mathbb{R}^m$ denotes the unknown latent quantity of interest that is governed by the PDE system of equation (2.1).

We proceed by approximating $\boldsymbol{s}(\boldsymbol{x})$ by a deep neural network $\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x})$, where $\boldsymbol{\theta}$ denotes all trainable parameters of the networks. Then, we can define the corresponding PDE residual as

$$\boldsymbol{r}_{\boldsymbol{\theta}}(\boldsymbol{x}, t) := \frac{\partial}{\partial t}\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}, t) + \mathcal{N}_{\boldsymbol{x}}[\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}, t)], \tag{2.4}$$

where the partial derivatives of the neural network representation with respect to space and time coordinates can be readily computed to machine precision using forward or reverse mode automatic differentiation [37]. A physics-informed neural network can be trained by minimizing the following composite loss function

$$\mathcal{L}(\boldsymbol{\theta}) = \lambda_r \mathcal{L}_r(\boldsymbol{\theta}) + \lambda_{\text{bc}} \mathcal{L}_{\text{bc}}(\boldsymbol{\theta}) + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}}(\boldsymbol{\theta}), \tag{2.5}$$

where

$$\mathcal{L}_r(\boldsymbol{\theta}) = \frac{1}{N_r} \sum_{i=1}^{N_r} \left| \boldsymbol{r}_{\boldsymbol{\theta}}(\boldsymbol{x}_r^i, t_r^i) \right|^2, \tag{2.6}$$

$$\mathcal{L}_{\text{bc}}(\boldsymbol{\theta}) = \frac{1}{N_{\text{bc}}} \sum_{i=1}^{N_{\text{bc}}} \left| \boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_{\text{bc}}^i, t_{\text{bc}}^i) - \boldsymbol{g}(\boldsymbol{x}_{\text{bc}}^i, t_{\text{bc}}^i) \right|^2, \tag{2.7}$$

$$\mathcal{L}_{\text{ic}}(\boldsymbol{\theta}) = \frac{1}{N_{\text{ic}}} \sum_{i=1}^{N_{\text{ic}}} \left| \boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_{\text{ic}}^i, 0) - \boldsymbol{u}(\boldsymbol{x}_{\text{ic}}^i) \right|^2. \tag{2.8}$$

Here, $N_r$, $N_{\text{bc}}$ and $N_{\text{ic}}$ denote the batch-sizes of the "training data" $\{(\boldsymbol{x}_r^i, t_r^i), \boldsymbol{f}(\boldsymbol{x}_r^i, t_r^i)\}_{i=1}^{N_r}$, $\{(\boldsymbol{x}_{\text{bc}}^i, t_{\text{bc}}^i), \boldsymbol{g}(\boldsymbol{x}_{\text{bc}}^i, t_{\text{bc}}^i)\}_{i=1}^{N_{\text{bc}}}$ and $\{\boldsymbol{x}_{\text{ic}}^i, \boldsymbol{h}(\boldsymbol{x}_{\text{ic}}^i)\}_{i=1}^{N_{\text{ic}}}$, respectively, which are randomly sampled in the computational domain and the boundary at each iteration of a stochastic gradient descent algorithm. Moreover, the parameters $\{\lambda_r, \lambda_{\text{bc}}, \lambda_{\text{ic}}\}$ correspond to weight coefficients in the loss function that can effectively assign a different learning rate to each individual loss term. These weights may be user-specified or tuned automatically during network training [15,16,38].

Despite a series of promising results [25,29,30,28], the original formulation of Raissi et al. [18] typically fails to handle long-time prediction tasks. To illustrate this, let us consider a simple gravity pendulum with damping governed by the following ODE system

$$\frac{ds_1}{dt} = s_2, \tag{2.9}$$

$$\frac{ds_2}{dt} = -\frac{b}{m} s_2 - \frac{g}{L} \sin(s_1), \tag{2.10}$$

for $t \in [0, T]$. The initial condition is given by $s_1(0) = s_2(0) = 1$. In this example, we take $m = L = 1$, $b = 0.05$ and $g = 9.81$. We are interested in using PINNs to solve this two-dimensional ODE system up to $T = 20$. To this end, we approximate the latent variables $s_1, s_2$ by a 5-layer fully-connected neural network $\boldsymbol{s}_{\boldsymbol{\theta}} = [s_{\boldsymbol{\theta}}^{(1)}, s_{\boldsymbol{\theta}}^{(2)}]$ with 100 units per hidden layer, and define the ODE residual as

$$r_{\boldsymbol{\theta}}^{(1)}(t) = \frac{ds_{\boldsymbol{\theta}}^{(1)}(t)}{dt} - s_{\boldsymbol{\theta}}^{(2)}(t), \tag{2.11}$$

$$r_{\boldsymbol{\theta}}^{(2)}(t) = \frac{ds_{\boldsymbol{\theta}}^{(2)}(t)}{dt} + \frac{b}{m} s_{\boldsymbol{\theta}}^2(t) - \frac{g}{L} \sin(s_{\boldsymbol{\theta}}^{(1)}(t)). \tag{2.12}$$

The corresponding PINNs loss function is given by

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_r(\boldsymbol{\theta}) + \mathcal{L}_{\text{ic}}(\boldsymbol{\theta}), \tag{2.13}$$

where

$$\mathcal{L}_r(\boldsymbol{\theta}) = \frac{1}{N_r} \sum_{i=1}^{N_r} \left[ \left| \boldsymbol{r}_{\boldsymbol{\theta}}^{(1)}(t_r^i) \right|^2 + \left| \boldsymbol{r}_{\boldsymbol{\theta}}^{(2)}(t_r^i) \right|^2 \right], \tag{2.14}$$

$$\mathcal{L}_{\text{ic}}(\boldsymbol{\theta}) = \left| s_{\boldsymbol{\theta}}^{(1)}(0) - s_1(0) \right|^2 + \left| s_{\boldsymbol{\theta}}^{(2)}(0) - s_2(0) \right|^2. \tag{2.15}$$

We set $N_r = 10^4$, and all collocation points $\{t_r^i\}_{i=1}^{N_r}$ are randomly sampled in $[0, T]$ at each iteration during training. We train the network by minimizing the loss function (2.13) for $10^5$ iterations of gradient descent using the Adam optimizer with default settings [39]. A comparison of the predicted solutions against their corresponding numerical estimation obtained with a conventional adaptive Runge-Kutta solver [3] is shown in Fig. 1. It is clear that the PINN model predictions collapse to zero after $T = 10$, which suggests that PINNs may be incapable of yielding accurate solutions for long-time integration problems.

There are some reasons that may explain the poor predictions and model collapse. One could be saturated activation functions due to large values of the input coordinates. Another possible reason may be the inability of neural networks

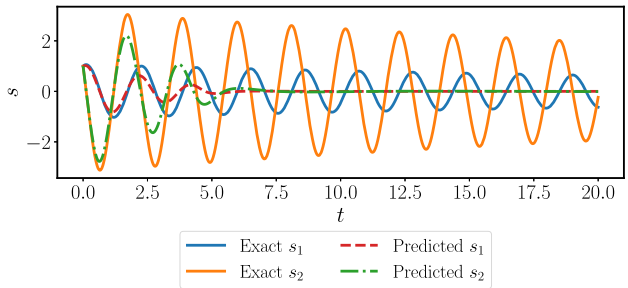


**Fig. 1.** *Gravity pendulum:* Predicted solution for $s_1(t)$ and $s_2(t)$ versus the corresponding reference solution. The result is obtained by training a conventional PINN (5 layers, 100 hidden units, tanh activations) for $10^5$ iterations of gradient descent using the Adam optimizer. Evidently, the model predictions collapse after $t = 10$.

to approximate high-frequency and complex functions because of spectral bias [40]. Admittedly, recent work has provided some remedies that can be directly applied to this case to improve model performance [41,17,42,21]. For example, to avoid saturation of activations, one can normalize the inputs such that they lie in a reasonable range, although this trick will lead to very small coefficients in the ODE system, and, consequently, to a singular perturbation problem [43]. Other approaches include, but are not limited to, using Fourier feature embeddings [41,17], as well as employing and training multiple individual networks in different temporal sub-domains [21,42,44]. The former typically requires some prior knowledge of the frequency content of the latent solution in order to properly initialize the model [17], while the latter approaches inevitably lead to a large computational cost. In the following sections, we present a simple yet effective strategy to solve long-time integration problems via physics-informed DeepONets [36].

## 3. Methods

### 3.1. A primer on physics-informed DeepONets

Recently, Lu et al. [45] proposed deep operator networks (DeepONets), which aim to learn abstract nonlinear operators mapping functions between infinite-dimensional Banach spaces. In follow up work, Wang et al. [36] developed physics-informed DeepONets, introducing an effective regularization mechanism for biasing the outputs of DeepONet models towards ensuring physical consistency. Here we present a brief overview of physics-informed DeepONets with a special focus on time-dependent PDEs (see equations (2.1)-(2.3)).

Let $\mathcal{U}, \mathcal{S}$ be two separate Banach spaces. We are interested in learning the solution operator $G$ from an initial condition $\boldsymbol{u} \in \mathcal{U}$ to the associated PDE solution $\boldsymbol{s}(\boldsymbol{x}, t) \in \mathcal{S}$. To this end, we represent the solution map $G$ by a DeepONet $G_{\boldsymbol{\theta}}$. As illustrated in Fig. 2, the DeepONet architecture consists of two separate neural networks referred to as the "branch net" and "trunk net", respectively. The branch net takes the parameters $\boldsymbol{s}$ as input and returns a feature embedding $[b_1, b_2, \dots, b_q]^T \in \mathbb{R}^q$ as output, where $\boldsymbol{u} = [\boldsymbol{u}(\boldsymbol{x}_1), \boldsymbol{u}(\boldsymbol{x}_2), \dots, \boldsymbol{u}(\boldsymbol{x}_m)]$ represents a function $\boldsymbol{u} \in \mathcal{U}$ evaluated at a collection of fixed locations $\{\boldsymbol{x}_i\}_{i=1}^m \subset \Omega$. The trunk net takes the continuous coordinates $(\boldsymbol{x}, t)$ as inputs, and outputs a feature embedding $[t_1, t_2, \dots, t_q]^T \in \mathbb{R}^q$. The final output of the DeepONet is obtained by merging the outputs of the branch and trunk networks via a dot product. More specifically, a DeepONet $G_{\boldsymbol{\theta}}$ prediction of an input function $\boldsymbol{u}$ evaluated at $(\boldsymbol{x}, t)$ can be expressed by

$$G_{\boldsymbol{\theta}}(\boldsymbol{u})(\boldsymbol{x}, t) = \sum_{k=1}^{q} \underbrace{b_k\left(\boldsymbol{u}\left(\boldsymbol{x}_1\right), \boldsymbol{u}\left(\boldsymbol{x}_2\right), \dots, \boldsymbol{u}\left(\boldsymbol{x}_m\right)\right)}_{\text{branch}} \underbrace{t_k(\boldsymbol{x}, t)}_{\text{trunk}}, \tag{3.1}$$

where $\boldsymbol{\theta}$ denotes the collection of all trainable weights and biases in the branch and trunk networks. A conventional DeepONet is a purely data-driven model which is trained by minimizing a supervised mean square error loss as follows

$$\mathcal{L}_{\text{data}}(\boldsymbol{\theta}) = \frac{1}{NP} \sum_{i=1}^{N} \sum_{j=1}^{P} \left| G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)})(\boldsymbol{x}_j^{(i)}, t_j^{(i)}) - G(\boldsymbol{u}^{(i)})(\boldsymbol{x}_j^{(i)}, t_j^{(i)}) \right|^2 \tag{3.2}$$

where $\{\boldsymbol{u}^{(i)}\}_{i=1}^N$ denotes $N$ separate input functions sampled from $\mathcal{U}$. For each $\boldsymbol{u}^{(i)}$, $\{\boldsymbol{x}_j^{(i)}, t_j^{(i)}\}_{j=1}^P$ are $P$ locations in the domain of $G(\boldsymbol{u}^{(i)})$, and $G(\boldsymbol{u}^{(i)})(\boldsymbol{x}_j^{(i)}, t_j^{(i)})$ are the corresponding measurements.

In comparison with conventional DeepONets, physics-informed DeepONets have the same network architecture while being trained by a hybrid loss function for fitting the observational data and enforcing the underlying PDE constraints. This is because a DeepONet is continuously differentiable with respect to the query points $(\boldsymbol{x}, t)$, thus allowing us to employ automatic differentiation [46,37] to compute the associated PDE residual

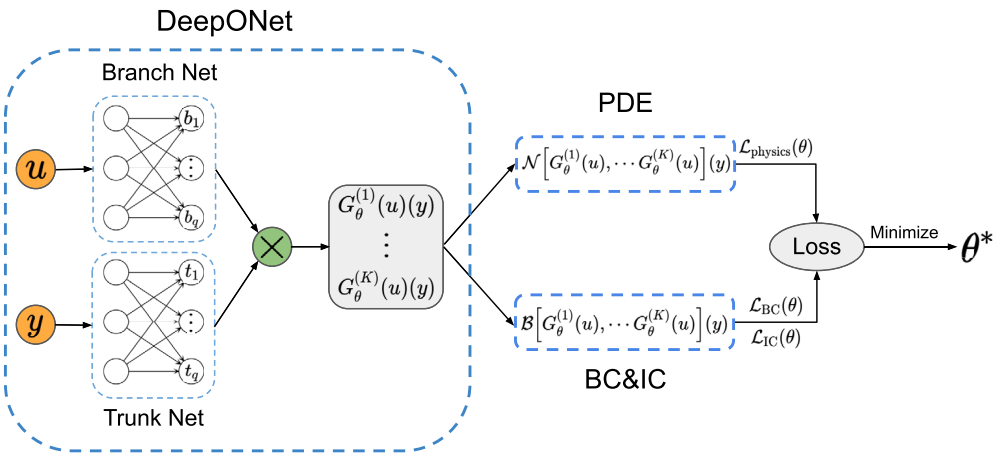


**Fig. 2.** *Making DeeoONets physics-informed:* The DeepONet architecture [45] consists of two sub-networks, the branch net for extracting latent representations of input functions, and the trunk net for extracting latent representations of input coordinates at which the output functions are evaluated. A continuous and differentiable representation of the output functions is then obtained by merging the latent representations extracted by each sub-network via a dot product. Automatic differentiation can then be employed to formulate appropriate regularization mechanisms for biasing the DeeoONet outputs to satisfy a given system of PDEs.

$$\mathcal{R}_{\boldsymbol{\theta}}[\boldsymbol{u}](\boldsymbol{x},t) = \frac{\partial G_{\boldsymbol{\theta}}(\boldsymbol{u})(\boldsymbol{x},t)}{\partial t} + \mathcal{N}_{\boldsymbol{x}}[G_{\boldsymbol{\theta}}(\boldsymbol{u})](\boldsymbol{x},t). \tag{3.3}$$

Then, we can construct a physics-informed DeepONet by formulating a composite loss function

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_{\text{data}}(\boldsymbol{\theta}) + \mathcal{L}_{\text{physics}}(\boldsymbol{\theta}) \tag{3.4}$$

where $\mathcal{L}_{\text{data}}$ is defined in equation (3.2) and

$$\mathcal{L}_{\text{physics}}(\boldsymbol{\theta}) = \frac{1}{NQ}\sum_{i=1}^{N}\sum_{j=1}^{Q}\left|\mathcal{R}_{\boldsymbol{\theta}}[\boldsymbol{u}^{(i)}](\boldsymbol{x}_{\text{r},j}^{(i)}, t_{\text{r},j}^{(i)})\right|^2. \tag{3.5}$$

Here $\{(\boldsymbol{x}_{\text{r},j}^{(i)}, t_{\text{r},j}^{(i)})\}_{j=1}^{Q}$ is a set of collocation points sampled from the computational domain $\Omega \times (0,T)$ for penalizing the PDE residuals. Thanks to this physics regularization, physics-informed DeepONets are much more data efficient than plain DeepONets. Even more interestingly, one can train a physics-informed DeepONet in a self-supervised manner without any observational data except for the given initial and boundary conditions. It is accomplished by minimizing the following purely physics-informed loss

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_{\text{ic}}(\boldsymbol{\theta}) + \mathcal{L}_{\text{bc}}(\boldsymbol{\theta}) + \mathcal{L}_{r}(\boldsymbol{\theta}), \tag{3.6}$$

where $\mathcal{L}_r(\boldsymbol{\theta})$ is the physics loss defined in equation (3.5) and

$$\mathcal{L}_{\text{ic}}(\boldsymbol{\theta}) = \frac{1}{NP}\sum_{i=1}^{N}\sum_{j=1}^{P}\left|G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)})(\boldsymbol{x}_{\text{ic},j}^{(i)}, 0) - \boldsymbol{u}^{(i)}(\boldsymbol{x}_{\text{ic},j}^{(i)})\right|^2 \tag{3.7}$$

$$\mathcal{L}_{\text{bc}}(\boldsymbol{\theta}) = \frac{1}{NP}\sum_{i=1}^{N}\sum_{j=1}^{P}\left|G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)})(\boldsymbol{x}_{\text{bc},j}^{(i)}, t_{\text{bc},j}^{(i)}) - \boldsymbol{g}(\boldsymbol{x}_{\text{bc},j}^{(i)}, t_{\text{bc},j}^{(i)})\right|^2. \tag{3.8}$$

Here $\{\boldsymbol{u}^{(i)}\}_{i=1}^{N}$ denotes $N$ separate input functions sampled from $\mathcal{U}$. For each $\boldsymbol{u}^{(i)}$, $\{(\boldsymbol{x}_{\text{ic},j}^{(i)}, 0\}_{j=1}^{P}$, $\{(\boldsymbol{x}_{\text{bc},j}^{(i)}, t_{\text{bc},j}^{(i)})\}_{j=1}^{P}$ are $P$ locations sampled from $\Omega \times \{T=0\}$ and $\partial\Omega \times [0,T]$ for enforcing the initial and boundary conditions, respectively. In contrast to the fixed sensor locations of $\{\boldsymbol{x}_i\}_{i=1}^{m}$, we remark that the locations of $\{(\boldsymbol{x}_{\text{ic},j}^{(i)}, 0\}_{j=1}^{P}$, $\{(\boldsymbol{x}_{\text{bc},j}^{(i)}, t_{\text{bc},j}^{(i)})\}_{j=1}^{P}$ and $\{(\boldsymbol{x}_{\text{r},j}^{(i)}, t_{\text{r},j}^{(i)})\}_{j=1}^{Q}$ may vary across different input samples $\boldsymbol{u}^{(i)}$.

### 3.2. Long-time integration of evolution equations

In this section, we present our main contribution for solving time-dependent PDEs (2.1)-(2.3) involving long-time integration using physics-informed DeepONets. Instead of decomposing the temporal domain $[0,T]$ into many sub-domains and sequentially solving each short-time problem with an independent neural network, we train a single physics-informed

DeepONet to learn the solution operator of the same PDE for a short time $t \in [0, \Delta t]$ subject to a distribution of initial conditions. Then we can obtain the inferred solution by recurrently replacing the initial condition with the model's prediction at $t = \Delta t$, and evaluating again the forward pass of the trained model. The details of the proposed strategy are summarized in Algorithm 1.

**Algorithm 1:** Long-time integration of evolution equations with physics-informed DeepONets.

Suppose that $G_{\boldsymbol{\theta}}$ is a trained physics-informed DeepONet that learned the solution operator of the time-dependent time PDE (2.1)-(2.3) for $t \in [0, \Delta t]$. Let $\{(\boldsymbol{x}_i, t_i)\}_{i=1}^{P}$ be a set of uniform grid points in $\Omega \times [0, \Delta t]$ and initialize the input function by $\boldsymbol{u}^0(\boldsymbol{x}) = \boldsymbol{u}(x)$ (the initial condition in equation (2.3)).

**for** $k = 1, \ldots, N$ **do**

(a) Infer the solution by running the forward pass of the physics-informed DeepONet

$$\boldsymbol{s}^k(\boldsymbol{x}_i, t_i) = G_{\boldsymbol{\theta}}(\boldsymbol{u}^{k-1})(\boldsymbol{x}_i, t_i), \quad \text{for } i = 1, 2, \ldots, P. \tag{3.9}$$

(b) Update the input function by

$$\boldsymbol{u}^k(\boldsymbol{x}) = \boldsymbol{s}^k(\boldsymbol{x}, \Delta t) \tag{3.10}$$

**end**

The final predicted solution in the whole domain $\Omega \times [0, T]$ can be obtained by concatenating all the inferred solutions $\{\boldsymbol{s}^k\}_{k=1}^{N}$.

To introduce more technical details, let us revisit the example of gravity pendulum presented in section 2 and pursue its simulation up to $T = 100$. Before doing so, notice that the output of a vanilla DeepONet is a scalar, while the solution of the ODE system in equations (2.9)-(2.10) is a 2-dimensional vector. To resolve this issue, we modify the forward pass (3.1) such that the DeepONet output can be a vector. Specifically, suppose that a DeepONet outputs $n$ different scalar functions. Then the forward pass of $i$-th function is given by

$$s_{\boldsymbol{\theta}}^{(i)} = G_{\boldsymbol{\theta}}^{(i)}(\boldsymbol{u})(\boldsymbol{x}, t) = \sum_{k=q_{i-1}+1}^{q_i} \underbrace{b_k\left(\boldsymbol{u}\left(\boldsymbol{x}_1\right), \boldsymbol{u}\left(\boldsymbol{x}_2\right), \ldots, \boldsymbol{u}\left(\boldsymbol{x}_m\right)\right)}_{\text{branch}} \underbrace{t_k(\boldsymbol{x}, t)}_{\text{trunk}} \tag{3.11}$$

for $i = 1, \ldots, n$ where $0 = q_0 < q_1 < \cdots < q_n = q$. With this generalized DeepONet architecture, one may note that the proposed algorithm can be in principle scaled to high dimensional problems.

For this 2D ODE, we take $n = 2$, $q = 200$ and $q_1 = 100$. Now we employ a DeepONet $G_{\boldsymbol{\theta}} = [G_{\boldsymbol{\theta}}^{(1)}, G_{\boldsymbol{\theta}}^{(2)}]$ to represent the solution map from initial conditions to the associated solutions in $[0, 1]$, where both the branch and trunk networks are 8-layer fully-connected neural networks with 100 units per hidden layer. Similar to equation (2.11)-(2.12), we can define the ODE residual for the physics-informed DeepONet model as

$$\mathcal{R}_{\boldsymbol{\theta}}^{(1)}[\boldsymbol{u}](t) = \frac{dG_{\boldsymbol{\theta}}^{(1)}(\boldsymbol{u})(t)}{dt} - G_{\boldsymbol{\theta}}^{(2)}(\boldsymbol{u})(t), \tag{3.12}$$

$$\mathcal{R}_{\boldsymbol{\theta}}^{(2)}[\boldsymbol{u}](t) = \frac{dG_{\boldsymbol{\theta}}^{(2)}(\boldsymbol{u})(t)}{dt} + \frac{b}{m} G_{\boldsymbol{\theta}}^{2}(\boldsymbol{u})(t) - \frac{g}{L} \sin(G_{\boldsymbol{\theta}}^{(1)}(\boldsymbol{u})(t)). \tag{3.13}$$

Then, the trainable parameters $\boldsymbol{\theta}$ can be optimized by minimizing the following loss

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_{\text{ic}}(\boldsymbol{\theta}) + \mathcal{L}_r(\boldsymbol{\theta}), \tag{3.14}$$

where

$$\mathcal{L}_{\text{ic}}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \left[ \left| G_{\boldsymbol{\theta}}^{(1)}(\boldsymbol{u}^{(i)})(0) - u_1^{(i)} \right|^2 + \left| G_{\boldsymbol{\theta}}^{(2)}(\boldsymbol{u}^{(i)})(0) - u_2^{(i)} \right|^2 \right] \tag{3.15}$$

$$\mathcal{L}_r(\boldsymbol{\theta}) = \frac{1}{NQ} \sum_{i=1}^{N} \sum_{j=1}^{Q} \left[ \left| \mathcal{R}_{\boldsymbol{\theta}}^{(1)}[\boldsymbol{u}^{(i)}](t_j^{(i)}) \right|^2 + \left| \mathcal{R}_{\boldsymbol{\theta}}^{(2)}[\boldsymbol{u}^{(i)}](t_j^{(i)}) \right|^2 \right]. \tag{3.16}$$

Here, $\boldsymbol{u}^{(i)} = [u_1^{(i)}, u_2^{(i)}]$ are the inputs of the branch network which denote the initial conditions. Moreover, for each $i$, $\{t_j^i\}_{j=1}^{Q}$ is a set of collocation points uniformly sampled from $[0, 1]$. In this example, we set $Q = 100$ and sample $N = 5 \times 10^4$ different initial conditions from a uniform distribution $\mathcal{U}(-3, 3)$. We train the physics-informed DeepONet for $3 \times 10^5$ iterations of gradient descent using Adam optimizer, and apply Algorithm 1 to the same initial condition as in the example presented in Fig. 1. As shown in Fig. 3, we observe that the model predictions are in good agreement with the exact solution. The resulting relative $L^2$ errors of $s^{(1)}$ and $s^{(2)}$ are 1.72% and 1.63%, respectively. It is worth emphasizing that the input function space $\mathcal{U}$ should be large enough to cover as many potential states of the underlying ODE/PDE system as possible.

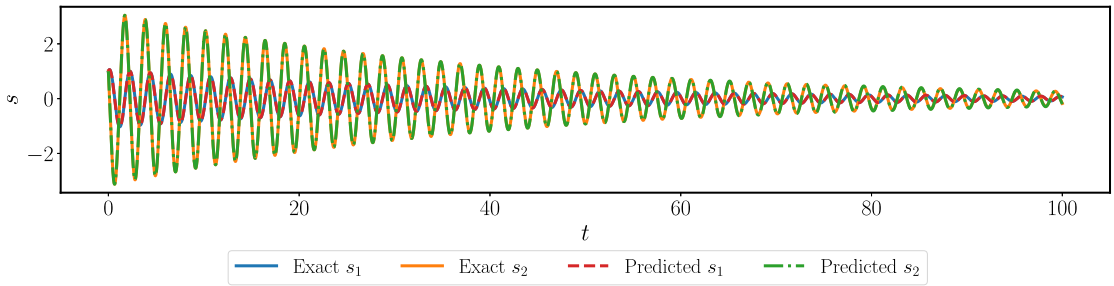


**Fig. 3.** *Gravity pendulum:* Predicted solution for $s_1(t)$ and $s_2(t)$ versus the corresponding reference solution. The result is obtained by applying Algorithm 1 to a trained a physics-informed DeepONet. The relative $L^2$ errors of $s_1$ and $s_2$ are 1.72% and 1.63%, respectively.

Otherwise, the trained model may not generalize very well for out-of-distribution initial conditions, possibly leading to large errors or even erroneous predictions.

## 4. Results

To demonstrate the effectiveness of the proposed algorithm, we provide a series of comprehensive numerical studies for solving various long-time integration problems. Throughout all benchmarks, we will employ a modified fully-connected neural network architecture [15] which has been empirically proved to outperform the standard fully-connected neural network architectures [15,36,27]. The forward pass is defined by

$$U = \phi(XW^1 + b^1), \quad V = \phi(XW^2 + b^2) \tag{4.1}$$

$$H^{(1)} = \phi(XW^{z,1} + b^{z,1}) \tag{4.2}$$

$$Z^{(k)} = \phi(H^{(k)}W^{z,k} + b^{z,k}), \quad k = 1, \dots, L \tag{4.3}$$

$$H^{(k+1)} = (1 - Z^{(k)}) \odot U + Z^{(k)} \odot V, \quad k = 1, \dots, L \tag{4.4}$$

$$f_{\boldsymbol{\theta}}(x) = H^{(L+1)}W + b, \tag{4.5}$$

where $X$ denotes the network inputs, and $\odot$ denotes element-wise multiplication. The parameters of this model are essentially the same as in conventional fully-connected architectures, with the addition of the weights and biases used by the two transformer sub-networks, i.e.,

$$\theta = \{W^1, b^1, W^2, b^2, (W^{z,l}, b^{z,l})_{l=1}^{L}, W, b\} \tag{4.6}$$

In all cases we employ hyperbolic tangent activation functions (Tanh) and initialize all trainable parameters using the Glorot normal scheme [47]. Physics-informed DeepONet models are trained via mini-batch gradient descent with a batch-size of 10,000 using the Adam optimizer with default settings [39], and an exponential learning rate decay with a decay-rate of 0.9 every 5,000 iterations. In this work, we tuned these hyper-parameters manually, without attempting to find the absolute best hyper-parameter setting. This process can be automated in the future leveraging effective techniques for meta-learning and hyper-parameter optimization [48]. Additional details related to performance metrics, computational cost, hyper-parameters and training details are discussed in the Supplementary Appendix. All results presented in this section can be reproduced using open-source code that will be made publicly available at https://github.com/PredictiveIntelligenceLab/Long-time-Integration-PI-DeepONets.

### 4.1. Inhomogeneous ODEs

As our first example, we start with a simple benchmark to illustrate how to generalize Algorithm 1 to inhomogeneous differential equations and non-autonomous cases. Particularly, we consider a 1D ODE of the form

$$\frac{ds}{dt} = 0.1 + \cos(t), \quad t \in [0, T], \tag{4.7}$$

$$s(0) = 0. \tag{4.8}$$

The objective is to use Algorithm 1 to learn the ODE solution for $T = 100$. However, one may note that the proposed algorithm cannot be directly applied to this problem because of the forcing term. For example, assume that $G_{\boldsymbol{\theta}}$ is a trained physics-informed DeepONet that approximates the solution operator from the initial condition to the associated solution of the ODE (4.7) for $t \in [0, 1]$ and $\tilde{s}(t)$ is an inferred solution corresponding to some initial condition. To obtain the solution in $[1, 2]$, the trained DeepONet is required to yield the predicted solution governed by the following ODE

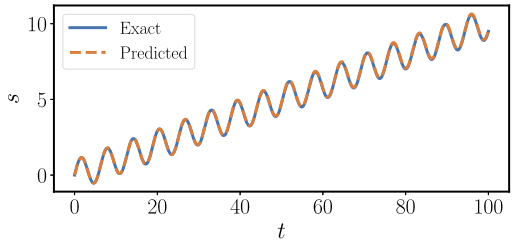


**Fig. 4.** *Inhomogeneous ODE:* Exact solution versus the predicted solution of a trained physics-informed DeepONet using Algorithm 1 to integrate the ODE system of equations (4.7)-(4.8) up to $T = 100$. The relative $L^2$ error is 0.26%.

$$\frac{ds}{dt} = 0.1 + \cos(t+1), \quad t \in [0, 1],$$
$$s(0) = \tilde{s}(1),$$

which is impossible because the trained model would be designing to work with a fixed forcing term. Fortunately, one can easily overcome this technical difficulty by solving the following parametric ODE

$$\frac{ds}{dt} = u(t), \quad t \in [0, \Delta t] \tag{4.9}$$
$$s(0) = u_0 \tag{4.10}$$

where both the initial condition and the forcing term can be considered as random inputs.

To represent the solution map of the above parametric ODE with a DeepONet $G_{\boldsymbol{\theta}}$, we employ a single trunk network for extracting latent representations of input coordinates, but two separate branch networks for representing the initial condition and the forcing term, respectively. The final output of the modified DeepONet architecture can be then obtained as

$$G_{\boldsymbol{\theta}}(\boldsymbol{u}, u_0)(t) = \sum_{k=1}^{q} \underbrace{b_k^{(1)}\left(u\left(t_1\right), u\left(t_2\right), \ldots, u\left(t_m\right)\right)}_{\text{branch I}} \cdot \underbrace{b_k^{(2)}\left(u_0\right)}_{\text{branch II}} \cdot \underbrace{t_k(t)}_{\text{trunk}}, \tag{4.11}$$

where $\{b_k^{(1)}\}_{k=1}^q, \{b_k^{(2)}\}_{k=1}^q$ are outputs of two branch networks, respectively, and $u_0$ denotes the input initial condition. Also, $\boldsymbol{u} = [u(t_1), u(t_2), \ldots, u(t_m)]$ represents the input forcing term evaluated at a set of fixed sensors $\{t_k\}_{k=1}^m \subset [0, \Delta t]$. In this example, we take $\Delta t = 1$, $m = 100$ and then $\{t_i\}_{i=1}^m$ are equi-spaced grid points in $[0, 1]$. Besides, all neural networks are 7-layer modified fully-connected neural networks (see equations (4.1)-(4.5)) with 100 units per hidden layer. The resulting physics-informed DeepONet can be trained by minimizing the following loss

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_{\text{ic}}(\boldsymbol{\theta}) + \mathcal{L}_r(\boldsymbol{\theta}), \tag{4.12}$$

where

$$\mathcal{L}_{\text{ic}}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N}\left|G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)}, u_0^{(i)})(0) - u_0^{(i)}\right|^2, \tag{4.13}$$

$$\mathcal{L}_r(\boldsymbol{\theta}) = \frac{1}{NQ}\sum_{i=1}^{N}\sum_{j=1}^{Q}\left|G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)}, u_0^{(i)})(t_j^{(i)}) - u^{(i)}(t_j^{(i)})\right|^2. \tag{4.14}$$

We generate "training data" by randomly sampling $N = 100$ inputs $\{\boldsymbol{u}^{(i)}\}_{i=1}^N, \{u_0^{(i)}\}_{i=1}^N$ from a Gaussian random field (GRF) with a length scale of $l = 0.5$ and an uniform distribution $\mathcal{U}(-0.1, 0.1)$ of initial conditions, respectively. For each paired input sample $(\boldsymbol{u}^{(i)}, u_0^{(i)})$, the collocation points $\{t_j^{(i)}\}_{j=1}^Q$ are uniformly sampled in the unit interval with $Q = 100$.

We train the physics-informed DeepONet for $1.5 \times 10^5$ iterations using the Adam optimizer with a learning rate decay by a factor of 0.9 for every 5,000 iterations. For every $10^4$ iterations, we augment the training data-set by adding 100 predicted solutions as the new initial conditions. Such an data augmentation will progressively shift an autonomous distribution to the desired distribution that generates the ground truth during training. Fig. 4 shows the inferred solution of the ODE system (4.7)-(4.8) obtained by Algorithm 1 and data augmentation during training. It can be observed that the model prediction achieves an excellent agreement with the exact solution as the resulting relative $L^2$ error over the whole time-domain is 0.26%. It is worth noting that the final solution is quite far from the original distribution $\mathcal{U}(-0.1, 0.1)$ for training, which suggests that the proposed data augmentation strategy can allow us to effectively tackle non-autonomous equations.

### 4.2. Stiff chemical kinetics

Our next example aims to demonstrate the capability of the proposed algorithm to perform long-time integration of stiff ODEs, which are generally hard to solve by conventional PINNs [49], as well as classical numerical methods [50,3]. To this end, we consider a classical stiff chemical kinetics problem describing the kinetics of an autocatalytic reaction [51,50]

$$\frac{ds_1}{dt} = -k_1 s_1 + k_3 s_2 s_3, \tag{4.15}$$

$$\frac{ds_2}{dt} = k_1 s_1 - k_2 s_2^2 - k_3 s_2 s_3, \tag{4.16}$$

$$\frac{ds_3}{dt} = k_2 s_2^2, \tag{4.17}$$

with the initial condition

$$s_1(0) = 1, \quad s_2(0) = s_3(0) = 0, \tag{4.18}$$

where $s_1, s_2, s_3$ denote the concentrations of different chemical species, while the reaction rate constants are $k_1 = 0.04, k_2 = 3 \times 10^4, k_3 = 10^7$. It worth pointing out that the different species have very different reaction timescales, especially for $s_2$, which consequently results in a very stiff system.

We proceed by employing a physics-informed DeepONet $\boldsymbol{G_\theta}$ to represent the solution operator $G$ mapping the initial condition $\boldsymbol{u} \in \mathbb{R}^3$ to the solution of the kinetic system in time interval $[0, 1]$ (i.e., $\Delta t = 1$). Since the ODE system has three variables, the forward pass of the DeepONet is given by equation (3.11) where we take $i = 3$ and $[q_0, q_1, q_2, q_3] = [0, 100, 200, 300]$. Accordingly, the ODE residuals are defined by

$$\mathcal{R}_{\boldsymbol{\theta}}^{(1)}[\boldsymbol{u}](t) = \frac{dG_{\boldsymbol{\theta}}^{(1)}(\boldsymbol{u})(t)}{dt} + k_1 G_{\boldsymbol{\theta}}^{(1)}(\boldsymbol{u})(t) - k_3 G_{\boldsymbol{\theta}}^{(2)}(\boldsymbol{u})(t) G_{\boldsymbol{\theta}}^{(3)}(\boldsymbol{u})(t), \tag{4.19}$$

$$\mathcal{R}_{\boldsymbol{\theta}}^{(2)}[\boldsymbol{u}](t) = \frac{dG_{\boldsymbol{\theta}}^{(2)}(\boldsymbol{u})(t)}{dt} - k_1 G_{\boldsymbol{\theta}}^{(1)}(\boldsymbol{u})(t) + k_2 [G_{\boldsymbol{\theta}}^{(2)}(\boldsymbol{u})(t)]^2 + k_3 G_{\boldsymbol{\theta}}^{(2)}(\boldsymbol{u})(t) G_{\boldsymbol{\theta}}^{(3)}(\boldsymbol{u})(t), \tag{4.20}$$

$$\mathcal{R}_{\boldsymbol{\theta}}^{(3)}[\boldsymbol{u}](t) = \frac{dG_{\boldsymbol{\theta}}^{(3)}(\boldsymbol{u})(t)}{dt} - k_2 [G_{\boldsymbol{\theta}}^{(2)}(\boldsymbol{u})(t)]^2, \tag{4.21}$$

where $\boldsymbol{G_\theta} = [G_{\boldsymbol{\theta}}^{(1)}, G_{\boldsymbol{\theta}}^{(2)}, G_{\boldsymbol{\theta}}^{(3)}]$ represents the three species $[s_1, s_2, s_3]$ in the kinetic system, and $\boldsymbol{u} = [u_1, u_2, u_3]$ denotes the initial concentration for each state variable. This allows us to formulate the physics-informed loss function

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_{\text{ic}}(\boldsymbol{\theta}) + \mathcal{L}_r(\boldsymbol{\theta}), \tag{4.22}$$

where

$$\mathcal{L}_{\text{ic}}(\boldsymbol{\theta}) = \sum_{k=1}^{3} \lambda_k \mathcal{L}_{\text{ic}}^{(k)}(\boldsymbol{\theta}) = \sum_{k=1}^{3} \lambda_k \left[ \frac{1}{N} \sum_{i=1}^{N} \left| G_{\boldsymbol{\theta}}^{(k)}(\boldsymbol{u}^{(i)})(0) - u_k^{(i)} \right|^2 \right], \tag{4.23}$$

$$\mathcal{L}_r(\boldsymbol{\theta}) = \sum_{k=1}^{3} \mathcal{L}_r^{(k)}(\boldsymbol{\theta}) = \sum_{k=1}^{3} \left[ \frac{1}{NQ} \sum_{i=1}^{N} \sum_{j=1}^{Q} \left| \mathcal{R}_{\boldsymbol{\theta}}^{(k)}[\boldsymbol{u}^{(i)}](t_j^{(i)}) \right|^2 \right]. \tag{4.24}$$

In this example, the branch and trunk networks parametrizing $\boldsymbol{G_\theta}$ are two 7-layer modified fully-connected neural networks with 100 neurons per hidden layer (see equations (4.1)-(4.5)). The model is trained on a data-set created by randomly sampling $N = 5 \times 10^4$ initial states $\{\boldsymbol{u}^{(i)}\}_{i=1}^N = \{[u_1^{(i)}, u_2^{(i)}, u_3^{(i)}]\}_{i=1}^N$, where $u_1^{(i)}, u_3^{(i)} \sim \mathcal{U}(0, 1)$, and $u_2^{(i)} \sim \mathcal{U}(0, 10^{-4})$, for all $i$. For each input sample $\boldsymbol{u}^i$ we have $Q = 10^3$, and $\{t_j^{(i)}\}_{j=1}^Q$ are collocation points sampled from $\mathcal{U}(0, 1)$. Furthermore, because the species $s_2$ is typically about four order of magnitudes smaller than $s_1$ and $s_3$, we manually re-scale the range of $G_{\boldsymbol{\theta}}^{(2)}$ by multiplying it by $10^{-4}$, and set $[\lambda_1, \lambda_2, \lambda_3] = [1, 10^6, 1]$ to the corresponding imbalance between the different loss functions. To generate the test data-set, we randomly sample 100 initial conditions, and obtain the corresponding numerical solutions in $[0, 500]$ by integrating the ODE using the Radau scheme [52].

We train the physics-informed DeepONet by minimizing the loss of equation (4.22) for $4 \times 10^5$ gradient descent iterations using the Adam optimizer. The inferred solution obtained using the trained model subject to the initial condition (4.18) is presented in the top panel of Fig. 5, from which we can observe an excellent agreement between the model inference and the ground truth with relative $L^2$ errors of 0.38%, 0.56%, 0.64% for $s_1, s_2, s_3$, respectively. Some representative visualizations for different initial conditions are shown in Supplementary Fig. 16. Moreover, the relative $L^2$ error of the model predictions, averaged over all 100 examples in the test data-set, is visualized in Fig. 6. From these figures, one may conclude that the trained model is capable of yielding accurate long-time predictions for different initial conditions.

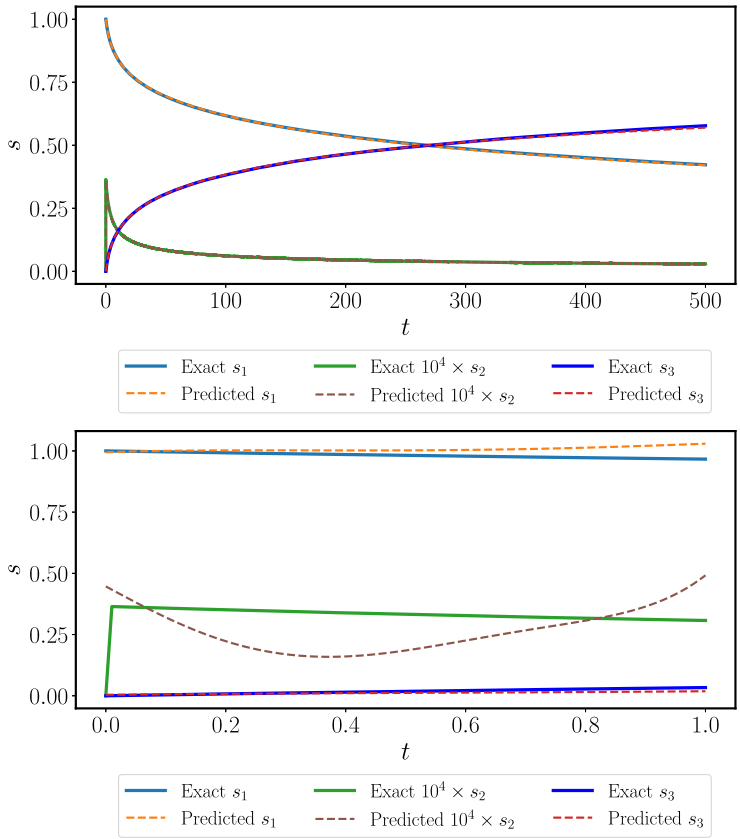


**Fig. 5.** *Stiff chemical kinetics: Top:* Reference solution versus the prediction of a trained scaled physics-informed DeepONet using Algorithm 1 for integrating the ODE system of equations (4.15)-(4.18) up to $T = 500$. The relative $L^2$ errors for $s_1, s_2, s_3$ are 0.38%, 0.56%, 0.64%, respectively. *Bottom:* Reference solution versus the prediction of a trained un-scaled physics-informed DeepONet. The relative $L^2$ errors for $s_1, s_2, s_3$ are 3.05%, 37.01%, 41.93%, respectively. We can observe that the trained model fails to yield accurate predictions even for a short-time interval corresponding to $T = 1$.

To demonstrate the necessity of assigning weights to the different terms in the physics-informed loss function, we train the same model without any weights under exactly the same hyper-parameter settings, and the result is summarized in the bottom panel of Fig. 5. It is evident that the un-scaled physics-informed DeepONet fails to learn the correct solution even on a unit interval, let alone long-time integration. These observations are consistent with the findings reported in [15,16,36], highly suggesting that rescaling the network outputs and the loss functions is a prerequisite for achieving good predictive accuracy, especially for problems that exhibit stiff and multi-scale behavior. Although here we manually assign weights assuming some prior knowledge of the underlying ODE model form, it must be emphasized that such a manual approach is time-consuming or even impractical for many realistic scenarios involving high-dimensional state spaces, multi-physics, complicated loss functions, etc. Hence, we point out that there is an urgent need to understand the training dynamics of physics-informed DeepONets and develop effective training algorithms that can automatically balance the interplay between different terms in the corresponding loss functions.

Once trained, the model can be rapidly queried to return prediction at short-time intervals. Each query typically takes $\mathcal{O}(10^{-3}$ ms) on a single Nvidia V100 GPU. Constructing the global PDE/ODE solution in large temporal domains using Algorithm 1 requires $N \sim \mathcal{O}(10) - \mathcal{O}(10^3)$ evaluations of the trained model, typically leading to a total inference time of $\sim \mathcal{O}(10^{-2})$ sec for a given initial condition. Multiple initial conditions can also be simulated at once, as our JAX [53] implementation is trivial vectorized and parallelized on GPU hardware. As demonstrated in Fig. 6, the proposed framework can perform long-time integration of more than 1,000 initial conditions in $\mathcal{O}(1)$ second, yielding a ∼10x-50x speedup compared to a traditional numerical solver. Unlike traditional numerical solvers that are heavily specialized to a specific type of dynamic behavior, this cost remains nearly constant for all examples considered in this work, regardless of the ODE/PDE system that is simulated, as it merely amounts to the cost associated with evaluating the forward pass of the trained DeepONet model. Our main motivation behind presenting this comparison is to demonstrate the fact that, once the DeepONet is trained, it can be fast in producing long trajectory roll-outs for batches of new and previously unseen initial conditions. Given the inherently different nature and implementation aspects of deep learning vs classical numerical methods, we acknowledge that is challenging to conduct a fair comparison, and by all means we do not argue that DeepONets can compare with classical solvers in terms of their accuracy/cost trade-off.

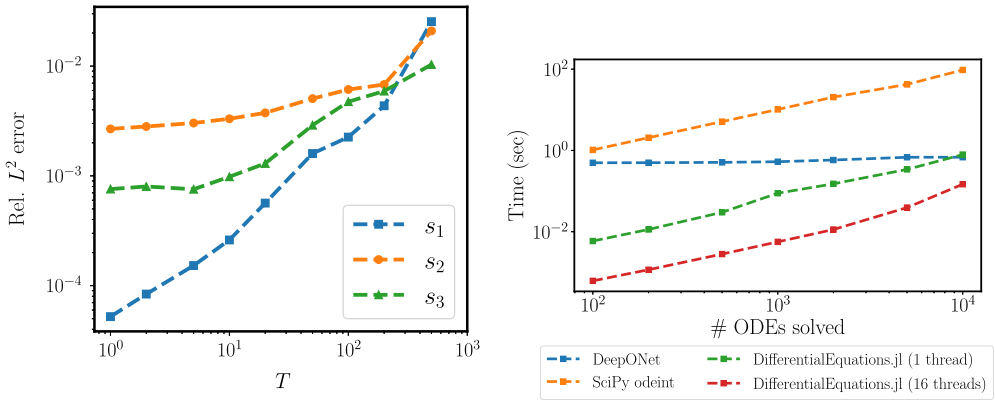


**Fig. 6.** *Stiff chemical kinetics: Left:* Relative $L^2$ prediction error of a trained physics-informed DeepONet as a function of the final prediction time $T$, averaged over 100 different examples in the test data-set. *Right:* Computational cost (sec) for performing inference with a trained physics-informed DeepONet model versus the time taken for solving the ODE with a conventional Radau scheme [52].

### 4.3. Wave propagation

In the numerical examples presented so far we mainly focus on ODEs. To highlight the ability of the proposed algorithm to handle long-time integration problems for PDEs, we begin by considering the 1D wave equation

$$\frac{\partial^2 s}{\partial t^2} = c^2 \frac{\partial^2 s}{\partial x^2}, \quad (x,t) \in [0,1] \times [0,T], \tag{4.25}$$

$$s(0,t) = s(1,t) = 0, \quad t \in [0,T], \tag{4.26}$$

$$s(x,0) = \sin(\pi x), \quad x \in [0,1], \tag{4.27}$$

$$\frac{\partial s}{\partial t}(x,0) = 0, \tag{4.28}$$

where we take $c = 1$ and $T = 100$. The exact solution is given by

$$s(x,t) = \sin(\pi x)\cos(c\pi t). \tag{4.29}$$

To solve this problem, we parametrize the initial condition by a Gaussian random field with a length scale $l = 0.5$, and use a DeepONet $G_{\boldsymbol{\theta}}$ to represent the solution map from initial conditions to the associated PDE solutions. Here, the branch and trunk networks are 5-layer modified fully-connected neural networks (see equations (4.1)-(4.5)) with 200 units per hidden layer. The parameters of the physics-informed DeepONet can be trained by minimizing the following loss function

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_{\text{ic}}(\boldsymbol{\theta}) + \mathcal{L}_{\text{bc}}(\boldsymbol{\theta}) + \mathcal{L}_r(\boldsymbol{\theta}), \tag{4.30}$$

where

$$\mathcal{L}_{\text{ic}}(\boldsymbol{\theta}) = \frac{1}{NP}\sum_{i=1}^{N}\sum_{j=1}^{P}\left[\left|G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)})(x_{\text{ic},j}^{(i)},0) - u^{(i)}(x_{\text{ic},j}^{(i)})\right|^2 + \left|\frac{\partial G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)})}{\partial t}(x_{\text{ic},j}^{(i)},0)\right|^2\right], \tag{4.31}$$

$$\mathcal{L}_{\text{bc}}(\boldsymbol{\theta}) = \frac{1}{NP}\sum_{i=1}^{N}\sum_{j=1}^{P}\left[\left|G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)})(0,t_{\text{bc},j}^{(i)})\right|^2 + \left|G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)})(1,t_{\text{bc},j}^{(i)})\right|^2\right], \tag{4.32}$$

$$\mathcal{L}_r(\boldsymbol{\theta}) = \frac{1}{NQ}\sum_{i=1}^{N}\sum_{j=1}^{Q}\left|\frac{\partial^2 G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)})}{\partial t^2}(x_{r,j}^{(i)},t_{r,j}^{(i)}) - c^2\frac{\partial^2 G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)})}{\partial x^2}(x_{r,j}^{(i)},t_{r,j}^{(i)})\right|^2. \tag{4.33}$$

Here, we take $\Delta t = 1$, $N = 10^4$, $m = P = 100$ and $Q = 200$. In particular, for every input sample $\boldsymbol{u}^{(i)}$, $\{x_{\text{ic},j}^{(i)}\}_{j=1}^{P}$, $\{t_{\text{bc},j}^{(i)}\}_{j=1}^{P}$ and $\{x_{r,j}^{(i)}, t_{r,j}^{(i)}\}_{j=1}^{Q}$ are uniformly sampled in the computational domain $[0,1]\times[0,1]$ for enforcing the initial/boundary conditions and the PDE residual, respectively. We train the physics-informed DeepONet for $2\times 10^5$ iterations of gradient descent using the Adam optimizer. A comparison of the long-time prediction against the ground truth is shown in Fig. 7. We can observe a good agreement between the predicted and the exact solution, yielding a relative error of $1.57e-02$ in the relative $L^2$ norm.

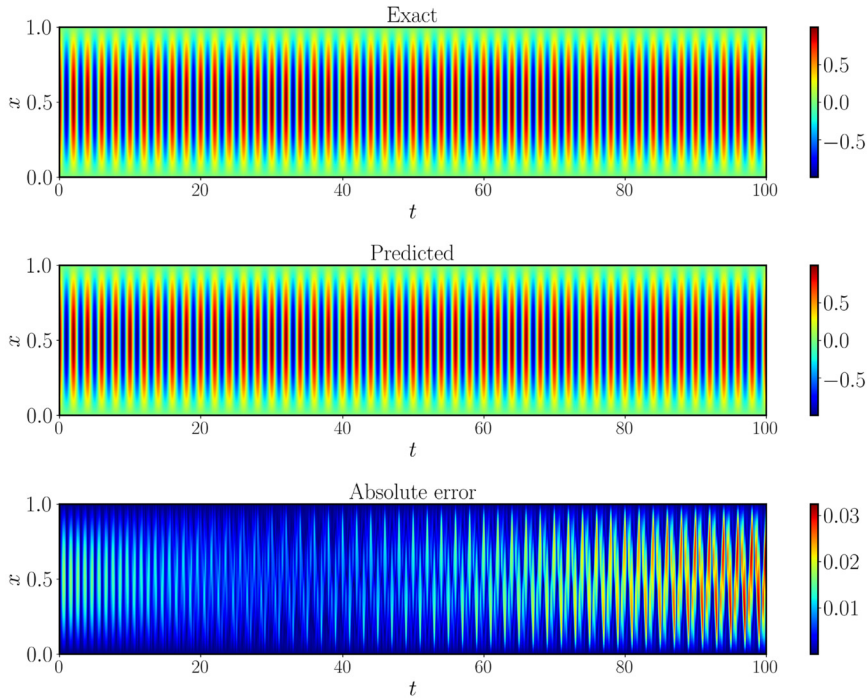


**Fig. 7.** *Wave propagation:* Comparison between the exact and the predicted solution of a trained physics-informed DeepONet using Algorithm 1 for integrating the PDE (4.25) - (4.28) up to $T = 100$. The relative $L^2$ error is 1.57%.

### 4.4. Diffusion-reaction dynamics

Next, we present a study on the effect of $\Delta t$ in Algorithm 1 by solving a long-time integration problem involving nonlinear diffusion-reaction dynamics, as described by the following PDE system

$$\frac{\partial s}{\partial t} = D\frac{\partial^2 s}{\partial x^2} + ks^2, \quad (x,t) \in (0,1] \times (0,T], \tag{4.34}$$

$$s(x,0) = u(x), \quad x \in [0,1] \tag{4.35}$$

$$s(0,t) = s(1,t) = 0, \quad t \in [0,T], \tag{4.36}$$

where $T = 50$, $D = 0.001$ is the diffusion coefficient, and $k = 0.001$ is the reaction rate. According to Algorithm 1, it suffices to learn the solution operator that maps the initial condition to the associated PDE solution in $[0, \Delta t]$. To this end, we approximate the operator by a DeepONet $G_{\boldsymbol{\theta}}$ where the branch net and the trunk network are 5-layer modified fully-connected neural network (see equations (4.1)-(4.5)) with 100 units per hidden layer. The physics-informed loss function is given by

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_{\text{ic}}(\boldsymbol{\theta}) + \mathcal{L}_{\text{bc}}(\boldsymbol{\theta}) + \mathcal{L}_r(\boldsymbol{\theta}), \tag{4.37}$$

where

$$\mathcal{L}_{\text{ic}}(\boldsymbol{\theta}) = \frac{1}{NP}\sum_{i=1}^{N}\sum_{j=1}^{P}\left|G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)})(x_{\text{ic},j}^{(i)}, 0) - u^{(i)}(x_{\text{ic},j}^{(i)})\right|^2, \tag{4.38}$$

$$\mathcal{L}_{\text{bc}}(\boldsymbol{\theta}) = \frac{1}{NP}\sum_{i=1}^{N}\sum_{j=1}^{P}\left[\left|G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)})(0, t_{\text{bc},j}^{(i)})\right|^2 + \left|G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)})(1, t_{\text{bc},j}^{(i)})\right|^2\right], \tag{4.39}$$

$$\mathcal{L}_r(\boldsymbol{\theta}) = \frac{1}{NQ}\sum_{i=1}^{N}\sum_{j=1}^{Q}\left|\frac{\partial G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)})}{\partial t}(x_{r,j}^{(i)}, t_{r,j}^{(i)}) - D\frac{\partial^2 G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)})}{\partial x^2}(x_{r,j}^{(i)}, t_{r,j}^{(i)}) - kG_{\boldsymbol{\theta}}^2(\boldsymbol{u}^{(i)})(x_{r,j}^{(i)}, t_{r,j}^{(i)})\right|. \tag{4.40}$$

Here, $\boldsymbol{u}^{(i)}$ denotes initial conditions randomly sampled from a GRF with a length scale of $l = 0.2$, and for each input sample, $\{x_{\text{ic},j}^{(i)}\}_{j=1}^{P}$, $\{t_{\text{bc},j}^{(i)}\}_{j=1}^{P}$ and $\{x_{r,j}^{(i)}, t_{r,j}^{(i)}\}_{j=1}^{Q}$ are uniformly sampled in the computational domain $[0,1] \times [0, \Delta t]$. In this example,

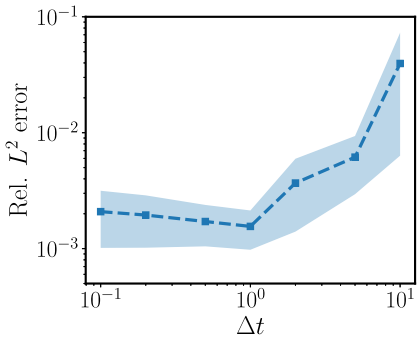


**Fig. 8.** *Diffusion-reaction dynamics:* Relative $L^2$ prediction errors of trained physics-informed DeepONets for different $\Delta t \in [0.1, 10]$, averaged over 100 different examples in the test data-set.

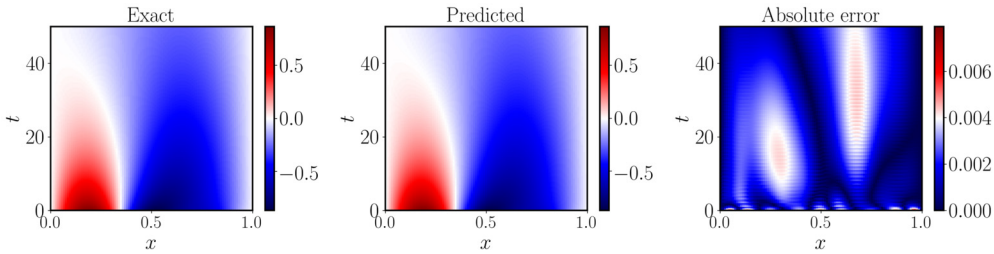


**Fig. 9.** *Diffusion-reaction dynamics:* Reference solution versus the predicted solution of a trained physics-informed DeepONet using Algorithm 1 for a representative input sample in the test data-set. The relative $L^2$ prediction error is 0.67%.

we take $N = 10^4$ and $m = P = Q = 100$. To generate the test data-set, we sample $N = 100$ input functions $u(x)$ from the same GRF and solve the diffusion-reaction system in $[0, 1] \times [0, T]$ by a second-order implicit finite difference method.

For different $\Delta t$, we train the physics-informed DeepONet for $2 \times 10^5$ iterations under exactly the same hyper-parameter settings. Fig. 8 shows the relative $L^2$ prediction errors of the trained models averaged over all examples in the test data-set. A key observation is that our approach is relatively robust against the choice of the time step size $\Delta t$, while trained model corresponding to $\Delta t = 1$ seems to yield the best predictive accuracy. Moreover, the predicted solution of the best trained model for one representative input sample is shown in Fig. 9. One can see that the predictions achieve an excellent agreement with the corresponding numerical estimations. The resulting relative $L^2$ error is 0.67%. Additional representative results corresponding to different initial conditions can be found in Supplementary Fig. 19, which further verifies our conclusion.

### *4.5. Korteweg–De Vries equation*

In our last example we would like to emphasize the effectiveness of physics-informed DeepONets in integrating observational data and governing evolution equations in a data-efficient manner. To this end, we pursue the long-time prediction of traveling solitons governed by the Korteweg–De Vries (KDV) equation, assuming that, instead of boundary conditions for the latent solution, only some sparse observations in a short-time interval are available. Specifically, we set $\epsilon = 1.2 \times 10^{-1}$ and $\mu = 8 \times 10^{-4}$ and consider the following PDE system

$$\frac{\partial s}{\partial t} + \epsilon \frac{\partial s}{\partial x} s + \mu \frac{\partial^3 s}{\partial x^3} = 0, \quad x \in [0, 5], \tag{4.41}$$

subject to the initial condition $s(x, 0) = u(x)$. For single solitons, the exact solution can be derived as [54]

$$s(x, t) = \frac{c}{2} \operatorname{sech}^2 \left[ \frac{\sqrt{c}}{2} (5x - \frac{1}{10} ct - a) \right] \tag{4.42}$$

where $c$ denotes the travel speed of a wave and $a$ determines the initial wave position in the physical domain. It is well-known that the KDV equation is one of the most important non-linear PDEs, originally derived to model shallow water waves and then used to describe a diversity of important finite amplitude dispersive wave phenomena in physics, such as acoustic waves in a harmonic crystal and ion-acoustic waves in plasmas [55]. Using Algorithm 1, the problem can be reduced to learning the solution operator $G$ mapping initial conditions to the corresponding PDE solutions in $[0, \Delta t]$.

We proceed by approximating $G$ with a DeepONet $G_\theta$, where both the branch and trunk networks are 7-layer modified fully-connected neural networks (see equations (4.1)-(4.5)) with 200 neurons per hidden layer. This allows us to define the corresponding PDE residual

$$\mathcal{R}[\boldsymbol{u}](x,t) = \frac{\partial G_{\boldsymbol{\theta}}(\boldsymbol{u})}{\partial t}(x,t) + \epsilon G_{\boldsymbol{\theta}}(\boldsymbol{u})(x,t)\frac{\partial G_{\boldsymbol{\theta}}(\boldsymbol{u})}{\partial x}(x,t) + \mu \frac{\partial^3 G_{\boldsymbol{\theta}}(\boldsymbol{u})}{\partial x^3}(x,t), \tag{4.43}$$

used to formulate the following loss function

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_{\text{data}}(\boldsymbol{\theta}) + \mathcal{L}_{\text{physics}}(\boldsymbol{\theta}), \tag{4.44}$$

where

$$\mathcal{L}_{\text{data}}(\boldsymbol{\theta}) = \frac{1}{NP}\sum_{i=1}^{N}\sum_{j=1}^{P}\left|G_{\boldsymbol{\theta}}(\boldsymbol{u}^{(i)})(x_{s,j}^{(i)}, t_{s,j}^{(i)}) - s^{(i)}(x_{s,j}^{(i)}, t_{s,j}^{(i)})\right|,$$

$$\mathcal{L}_{\text{physics}}(\boldsymbol{\theta}) = \frac{1}{NQ}\sum_{i=1}^{N}\sum_{j=1}^{Q}\left|\mathcal{R}[\boldsymbol{u}^{(i)}](x_{j}^{(i)}, t_{j}^{(i)}) - s^{(i)}(x_{r,j}^{(i)}, t_{r,j}^{(i)})\right|.$$

Here, $\boldsymbol{u}^{(i)} = [u^{(i)}(x_1), u^{(i)}(x_2), \ldots, u^{(i)}(x_m)]$ denotes the initial conditions evaluated at a set of fixed sensor locations $\{x_i\}_{i=1}^{m}$, and $s^{(i)}$ is the associated PDE solution corresponding to each $u^{(i)}$. For every $i$, $\{(x_{s,j}^{(i)}, t_{s,j}^{(i)}), s^{(i)}(x_{s,j}^{(i)}, t_{s,j}^{(i)})\}_{j=1}^{P}$ are available solution measurements, while $\{(x_j^{(i)}, t_j^{(i)}) - s^{(i)}(x_{r,j}^{(i)}, t_{r,j}^{(i)})\}_{j=1}^{Q}$ is a set of collocation points randomly sampled in the computational domain $[0,5]\times[0,\Delta t]$ for imposing the PDE constraint. To obtain a set of training data, we sample pairs $(a,c)$ where $a \sim \mathcal{U}(0,5), b \sim \mathcal{U}(0,1)$, and randomly selecting $P$ observations of the exact solution $s(x,t)$ using equation (4.42). In this example, we set $N = 5\times 10^3$, $\Delta t = 10$ and $m = P = Q = 200$.

We train the proposed physics-informed DeepONet using the Adam optimizer for $2\times 10^5$ iterations, and then apply Algorithm 1 to the initial condition with $a = 1, c = 3/2$, i.e.

$$s(x,0) = \frac{3}{4}\,\text{sech}^2\left[\frac{\sqrt{3/2}}{2}(5x-1)\right].$$

The top panel of Fig. 10 shows the predicted global spatio-temporal solution, for which the resulting prediction error is measured at $1.24e-02$ in the relative $L^2$-norm. A more detailed assessment of the predicted solution is presented in the bottom panel of Fig. 10, which displays a comparison between the exact and the predicted solutions at different temporal snapshots $t = 0, 50, 100$. Through domain decomposition in time, the physics-informed DeepONet can accurately predict the traveling wave across a long-time horizon. We further investigate the performance of a conventional DeepONet model, which can be trained by minimizing the loss function $\mathcal{L}_{\text{data}}$ solely. As shown in Fig. 11, the results of the vanilla DeepONet are slightly worse than the physics-informed DeepONet. This conclusion is further confirmed by a relatively large prediction error of $6.84e-02$. For a more thorough comparison, we train both a vanilla DeepONet and a physics-informed DeepONet for different number of training data points (i.e., different number of samples $u$), and report the mean of the relative $L^2$ error of $s(x,t)$ over all 100 examples in the test data-set in Fig. 12. Compared to vanilla DeepONets, the proposed physics-informed DeepONets can not only achieve better predictive accuracy, but also utilize the observed data more efficiently, therefore providing enhanced effectiveness in small data regime.

## 5. Discussion

We have presented an effective deep learning framework for performing long-time integration of evolution equations; a task for which many popular ML-based approaches struggle to yield accurate results (see SI section S5 for additional motivation and results). The proposed approach avoids the high computational cost associated with training multiple networks [21,42,44,14], as well as the cost associated with the requirement of generating large training data-sets [56]. The proposed methodology can be viewed as a new effective way of performing temporal domain decomposition, in which a single network needs to be trained only within a short-time interval, albeit across a distribution of initial conditions. Leveraging the framework of physics-informed DeepONets [36], we put forth a simple two-step process. First, we demonstrate how deep neural networks can parametrize and learn solution operators that map initial conditions to associated ODE/PDE solutions in a short-time interval. The trained model can be then iteratively evaluated to construct the global ODE/PDE in large temporal domains, at any arbitrary spatio-temporal resolution, and across a range of initial conditions. The effectiveness and robustness of the proposed algorithms has been demonstrated in a series of detailed numerical experiments involving long-time simulation of evolution laws that describe inherently different physical processes, including wave propagation, reaction-diffusion dynamics, and stiff chemical kinetics.

Long-time integration is often one of the main bottlenecks in simulating complex multi-scale and multi-physics processes in science and engineering, with applications ranging from understanding the effect of anthropogenic pollution in ocean and atmospheric chemical transport, to designing fuel-efficient combustion engines, to elucidating the biophysical mechanisms underpinning cardiovascular disease, and beyond. Developing high-fidelity simulation tools for such problems is technically and computationally challenging due to the long-time duration of interest, the stochastic nature of fragmentation and turbulent mixing phenomena, the temperature dependency of thermal neutralization mechanisms, and the stiff dynamics

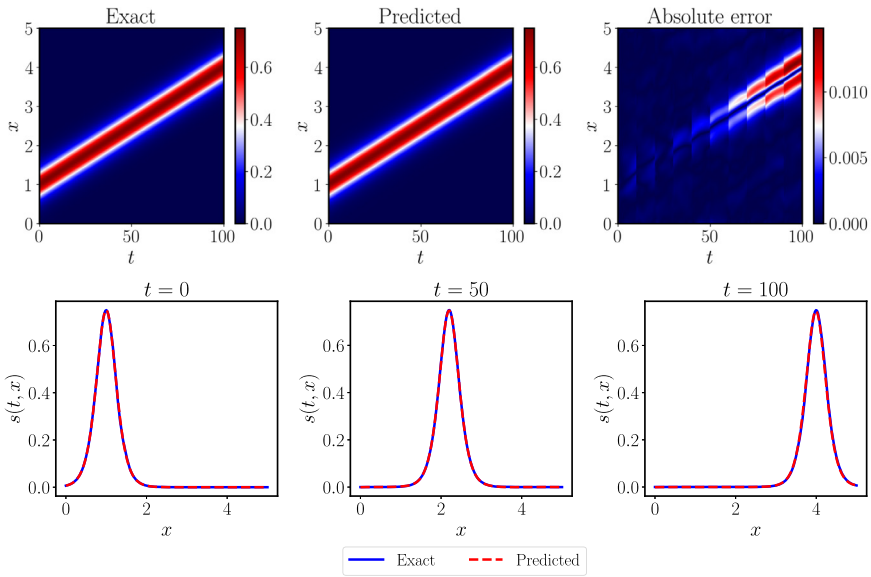


**Fig. 10.** *Korteweg–De Vries equation: Top:* Exact solution versus the predicted solution of a trained physics-informed DeepONet using Algorithm 1 for integrating the PDE system (4.41) up to $T = 100$. The relative $L^2$ error is 1.24%. *Bottom:* Comparison between the exact and the predicted solutions at different temporal snapshots corresponding to $t = 0, 50, 100$.

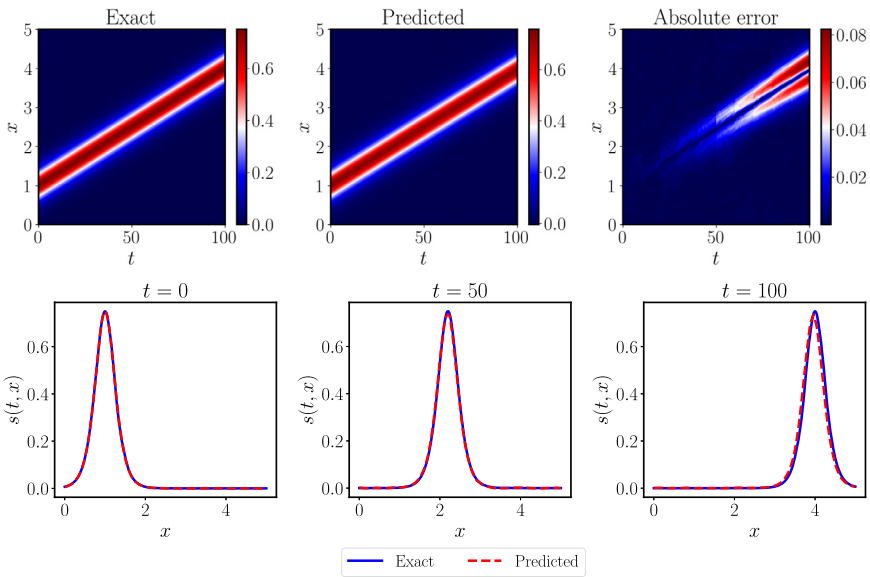


**Fig. 11.** *Korteweg–De Vries equation: Top:* Exact solution versus the predicted solution of a trained conventional DeepONet using Algorithm 1 for integrating the PDE system (4.41) up to $T = 100$. The relative $L^2$ error is 6.84%. *Bottom:* Comparison between the exact and the predicted solutions at different temporal snapshots corresponding to $t = 0, 50, 100$.

of bio-chemical reactions. The methods presented in this work are a first step towards demonstrating feasibility for ML-based techniques in reducing computational costs and enabling the rapid and accurate emulation of such non-equilibrium processes in science and engineering.

Despite the early promise demonstrated here, we have to admit that we are still at the very early stages of tackling long-time prediction problems with physics-informed DeepONets. There are many open questions worth considering as future research directions. From a theoretical point of view, it is important to develop a better understanding of how approximation errors affect the stability and accuracy of the proposed methods. This is a crucial element in applications that demand accuracy and convergence guarantees, where currently classical numerical solvers remain the de-facto choice. Recently, Wang et al. proposed causal training strategy [57], showing some promising results in simulating chaotic Kuramoto-Sivashinsky [58,59] and Navier-Stokes equations with physics-informed neural networks. Therefore, from a practical point of view, it would be intriguing to apply the proposed approach with the causal training to emulate high-dimensional and chaotic dy-

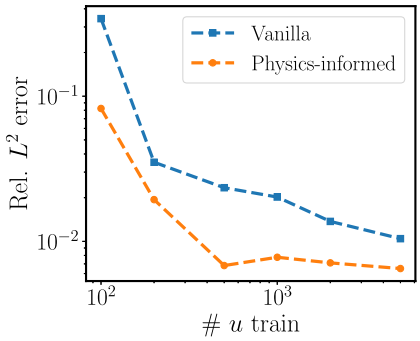


**Fig. 12.** Relative $L^2$ prediction error of a vanilla DeepONet [45] versus the proposed physics-informed DeepONet as a function of the number of examples in the training data-set.

namical systems; tasks that still remain elusive to ML-based techniques. These problems typically involve fast transitions of frequencies and are extremely sensitive to initial conditions, which inevitably introduces great challenges to both traditional numerical solvers, as well as ML-based approaches. From a methodology point of view, our algorithm can be regarded as temporal "domain decomposition" technique, which further motivates us to develop similar domain decomposition strategies for spatial domains. This may be combined with traditional domain decomposition techniques such as the Schwartz alternating method [60] to open the path of scaling physics-informed ML approaches to large computational domains and complex geometries [61]. We believe that addressing these open questions will pave a new way to developing scientific machine learning algorithms with better robustness and accuracy guarantees, as needed for many critical applications in computational science and engineering.

## CRediT authorship contribution statement

**Sifan Wang:** Conceptualization, Implementation, Simulation, Data curation, Visualization, Writing – Original draft preparation. **Paris Perdikaris:** Conceptualization, Supervision, Funding, Writing – Reviewing and Editing.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

Data will be made available on request.

## Acknowledgements

This work received support from DOE grant DE-SC0019116, AFOSR grant FA9550-20-1-0060, and DOE-ARPA grant DE-AR0001201. We would also like to thank the developers of the software that enabled our research, including JAX [53], Matplotlib [62], NumPy [63] and DeepXDE [64]. We would also like to thank Chris Rackauckas (MIT) for providing an optimized Julia implementation for a Radau solver [65] that helped us improve the comparative study presented in Fig. 6.

## Appendix A. Supplementary material

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.jcp.2022.111855.

## References

[1] Richard Courant, David Hilbert, Methods of Mathematical Physics: Partial Differential Equations, John Wiley & Sons, 2008.
[2] Parviz Moin, Fundamentals of Engineering Numerical Analysis, Cambridge University Press, 2010.
[3] Arieh Iserles, A First Course in the Numerical Analysis of Differential Equations, vol. 44, Cambridge University Press, 2009.
[4] Thomas J.R. Hughes, The Finite Element Method: Linear Static and Dynamic Finite Element Analysis, Courier Corporation, 2012.
[5] Yousef Saad, Iterative Methods for Sparse Linear Systems, SIAM, 2003.
[6] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, Liu Yang, Physics-informed machine learning, Nat. Rev. Phys. (2021) 1–19.
[7] Dimitris C. Psichogios, Lyle H. Ungar, A hybrid neural network-first principles approach to process modeling, AIChE J. 38 (10) (1992) 1499–1511.
[8] Isaac E. Lagaris, Aristidis Likas, Dimitrios I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, IEEE Trans. Neural Netw. 9 (5) (1998) 987–1000.

[9] Maziar Raissi, Paris Perdikaris, George Em Karniadakis, Inferring solutions of differential equations using noisy multi-fidelity data, J. Comput. Phys. 335 (2017) 736–746.
[10] Yifan Chen, Bamdad Hosseini, Houman Owhadi, Andrew M. Stuart, Solving and learning nonlinear PDEs with Gaussian processes, arXiv preprint, arXiv:2103.12959, 2021.
[11] Stefanie Gunther, Lars Ruthotto, Jacob B. Schroder, Eric C. Cyr, Nicolas R. Gauger, Layer-parallel training of deep residual neural networks, SIAM J. Math. Data Sci. 2 (1) (2020) 1–23.
[12] Bing Yu, et al., The deep Ritz method: a deep learning-based numerical algorithm for solving variational problems, Commun. Math. Stat. 6 (1) (2018) 1–12.
[13] Esteban Samaniego, Cosmin Anitescu, Somdatta Goswami, Vien Minh Nguyen-Thanh, Hongwei Guo, Khader Hamdia, X. Zhuang, T. Rabczuk, An energy approach to the solution of partial differential equations in computational mechanics via machine learning: concepts, implementation and applications, Comput. Methods Appl. Mech. Eng. 362 (2020) 112790.
[14] Maziar Raissi, Hessam Babaee, Peyman Givi, Deep learning of turbulent scalar mixing, Phys. Rev. Fluids 4 (12) (2019) 124501.
[15] Sifan Wang, Yujun Teng, Paris Perdikaris, Understanding and mitigating gradient pathologies in physics-informed neural networks, arXiv preprint, arXiv:2001.04536, 2020.
[16] Sifan Wang, Xinling Yu, Paris Perdikaris, When and why PINNs fail to train: a neural tangent kernel perspective, arXiv preprint, arXiv:2007.14527, 2020.
[17] Sifan Wang, Hanwen Wang, Paris Perdikaris, On the eigenvector bias of Fourier feature networks: from regression to solving multi-scale PDEs with physics-informed neural networks, arXiv preprint, arXiv:2012.10047, 2020.
[18] Maziar Raissi, Paris Perdikaris, George E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707.
[19] Ehsan Kharazmi, Zhongqiang Zhang, George Em Karniadakis, Variational physics-informed neural networks for solving partial differential equations, arXiv preprint, arXiv:1912.00873, 2019.
[20] Ameya D. Jagtap, George Em Karniadakis, Extended physics-informed neural networks (XPINNs): a generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations, Commun. Comput. Phys. 28 (5) (2020) 2002–2041.
[21] Xuhui Meng, Zhen Li, Dongkun Zhang, George Em Karniadakis, PPINN: parareal physics-informed neural network for time-dependent PDEs, Comput. Methods Appl. Mech. Eng. 370 (2020) 113250.
[22] Yinhao Zhu, Nicholas Zabaras, Phaedon-Stelios Koutsourelakis, Paris Perdikaris, Physics-constrained deep learning for high-dimensional surrogate modeling and uncertainty quantification without labeled data, J. Comput. Phys. 394 (2019) 56–81.
[23] Nicholas Geneva, Nicholas Zabaras, Modeling the dynamics of PDE systems with physics-constrained deep auto-regressive networks, J. Comput. Phys. 403 (2020) 109056.
[24] Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, Peter Battaglia, Learning to simulate complex physics with graph networks, in: International Conference on Machine Learning, PMLR, 2020, pp. 8459–8468.
[25] Maziar Raissi, Alireza Yazdani, George Em Karniadakis, Hidden fluid mechanics: learning velocity and pressure fields from flow visualizations, Science 367 (6481) (2020) 1026–1030.
[26] A.M. Tartakovsky, C. Ortiz Marrero, Paris Perdikaris, G.D. Tartakovsky, D. Barajas-Solano, Physics-informed deep neural networks for learning parameters and constitutive relationships in subsurface flow problems, Water Resour. Res. 56 (5) (2020) e2019WR026731.
[27] Oliver Hennigh, Susheela Narasimhan, Mohammad Amin Nabian, Akshay Subramaniam, Kaustubh Tangsali, Max Rietmann, Jose del Aguila Ferrandis, Wonmin Byeon, Zhiwei Fang, Sanjay Choudhry, NVIDIA SimNet$^{TM}$: an AI-accelerated multi-physics simulation framework, arXiv preprint, arXiv:2012.07938, 2020.
[28] Shengze Cai, Zhicheng Wang, Sifan Wang, Paris Perdikaris, George Em Karniadakis, Physics-informed neural networks for heat transfer problems, J. Heat Transf. 143 (6) (2021).
[29] Georgios Kissas, Yibo Yang, Eileen Hwuang, Walter R. Witschey, John A. Detre, Paris Perdikaris, Machine learning in cardiovascular flows modeling: predicting arterial blood pressure from non-invasive 4D flow MRI data using physics-informed neural networks, Comput. Methods Appl. Mech. Eng. 358 (2020) 112623.
[30] Francisco Sahli Costabal, Yibo Yang, Paris Perdikaris, Daniel E. Hurtado, Ellen Kuhl, Physics-informed neural networks for cardiac activation mapping, Front. Phys. 8 (2020) 42.
[31] Lu Lu, Ming Dao, Punit Kumar, Upadrasta Ramamurty, George Em Karniadakis, Subra Suresh, Extraction of mechanical properties of materials through deep learning from instrumented indentation, Proc. Natl. Acad. Sci. USA 117 (13) (2020) 7052–7062.
[32] Yuyao Chen, Lu Lu, George Em Karniadakis, Luca Dal Negro, Physics-informed neural networks for inverse problems in nano-optics and metamaterials, Opt. Express 28 (8) (2020) 11618–11633.
[33] Somdatta Goswami, Cosmin Anitescu, Souvik Chakraborty, Timon Rabczuk, Transfer learning enhanced physics informed neural network for phase-field modeling of fracture, Theor. Appl. Fract. Mech. 106 (2020) 102447.
[34] Dennis Elbrächter, Philipp Grohs, Arnulf Jentzen, Christoph Schwab, DNN expression rate analysis of high-dimensional PDEs: application to option pricing, arXiv preprint, arXiv:1809.07669, 2018.
[35] Jiequn Han, Arnulf Jentzen, E. Weinan, Solving high-dimensional partial differential equations using deep learning, Proc. Natl. Acad. Sci. USA 115 (34) (2018) 8505–8510.
[36] Sifan Wang, Hanwen Wang, Paris Perdikaris, Learning the solution operator of parametric partial differential equations with physics-informed DeepONets, arXiv preprint, arXiv:2103.10974, 2021.
[37] Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, Jeffrey Mark Siskind, Automatic differentiation in machine learning: a survey, J. Mach. Learn. Res. 18 (2018).
[38] Levi McClenny, Ulisses Braga-Neto, Self-adaptive physics-informed neural networks using a soft attention mechanism, arXiv preprint, arXiv:2009.04544, 2020.
[39] Diederik P. Kingma, Jimmy Ba, Adam: a method for stochastic optimization, arXiv preprint, arXiv:1412.6980, 2014.
[40] Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua Bengio, Aaron Courville, On the spectral bias of neural networks, in: International Conference on Machine Learning, 2019, pp. 5301–5310.
[41] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, Ren Ng, Fourier features let networks learn high frequency functions in low dimensional domains, arXiv preprint, arXiv:2006.10739, 2020.
[42] Colby L. Wight, Jia Zhao, Solving Allen-Cahn and Cahn-Hilliard equations using the adaptive physics informed neural networks, arXiv preprint, arXiv:2007.04542, 2020.
[43] E. Weinan, Principles of Multiscale Modeling, Cambridge University Press, 2011.
[44] Yifan Du, Tamer A. Zaki, Evolutional deep neural network, arXiv preprint, arXiv:2103.09959, 2021.
[45] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, George Em Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, Nat. Mach. Intell. 3 (3) (2021) 218–229.
[46] Andreas Griewank, et al., On automatic differentiation, Math. Program. 6 (6) (1989) 83–107.
[47] Xavier Glorot, Yoshua Bengio, Understanding the difficulty of training deep feedforward neural networks, in: Proceedings of the thirteenth international conference on artificial intelligence and statistics, 2010, pp. 249–256.

[48] Chelsea Finn, Pieter Abbeel, Sergey Levine, Model-agnostic meta-learning for fast adaptation of deep networks, in: International Conference on Machine Learning, PMLR, 2017, pp. 1126–1135.
[49] Weiqi Ji, Weilun Qiu, Zhiyu Shi, Shaowu Pan, Sili Deng, Stiff-PINN: physics-informed neural network for stiff chemical kinetics, arXiv preprint, arXiv: 2011.04520, 2020.
[50] Gerhard Wanner, Ernst Hairer, Solving Ordinary Differential Equations II, vol. 375, Springer, Berlin, Heidelberg, 1996.
[51] H.H. Robertson, The solution of a set of reaction rate equations, in: Numerical Analysis: An Introduction, 1966, 178182.
[52] Ernst Hairer, Gerhard Wanner, Stiff differential equations solved by Radau methods, J. Comput. Appl. Math. 111 (1–2) (1999) 93–111.
[53] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, Qiao Zhang, JAX: composable transformations of Python+NumPy programs, 2018.
[54] Klaus Brauer, The Korteweg-de Vries Equation: History, Exact Solutions, and Graphical Representation, University of Osnabrück, Germany, 2000.
[55] John W. Miles, The Korteweg-de Vries equation: a historical essay, J. Fluid Mech. 106 (1981) 131–147.
[56] Dmitrii Kochkov, Jamie A. Smith, Ayya Alieva, Qing Wang, Michael P. Brenner, Stephan Hoyer, Machine learning accelerated computational fluid dynamics. arXiv preprint, arXiv:2102.01010, 2021.
[57] Sifan Wang, Shyam Sankaran, Paris Perdikaris, Respecting causality is all you need for training physics-informed neural networks, arXiv preprint, arXiv:2203.07404, 2022.
[58] G.I. Sivashinsky, Nonlinear analysis of hydrodynamic instability in laminar flames—I. Derivation of basic equations, AcAau 4 (11) (1977) 1177–1206.
[59] Yoshiki Kuramoto, Diffusion-induced chaos in reaction systems, Prog. Theor. Phys. Suppl. 64 (1978) 346–367.
[60] Pierre-Louis Lions, On the Schwarz alternating method I, in: First International Symposium on Domain Decomposition Methods for Partial Differential Equations, vol. 1, Paris, France, 1988, p. 42.
[61] Hengjie Wang, Robert Planas, Aparna Chandramowlishwaran, Ramin Bostanabad, Train once and use forever: solving boundary value problems in unseen domains with pre-trained deep learning models, arXiv preprint, arXiv:2104.10873, 2021.
[62] John D. Hunter, Matplotlib: a 2D graphics environment, IEEE Ann. Hist. Comput. 9 (03) (2007) 90–95.
[63] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, et al., Array programming with NumPy, Nature 585 (7825) (2020) 357–362.
[64] Lu Lu, Xuhui Meng, Zhiping Mao, George Em Karniadakis, DeepXDE: a deep learning library for solving differential equations, SIAM Rev. 63 (1) (2021) 208–228.
[65] Christopher Rackauckas, Qing Nie, DifferentialEquations.jl—a performant and feature-rich ecosystem for solving differential equations in Julia, J. Open Res. Softw. 5 (1) (2017).